

GRADO EN CIENCIA DE DATOS



VNIVERSITAT
ID VALÈNCIA

TRABAJO FIN DE GRADO

CLASIFICACIÓN DE EEG REALES BASADA EN
TÉCNICAS CUÁNTICAS DE RESERVOIR
COMPUTING.

AUTOR:
MANUEL PÉREZ PERDOMO

TUTORES:
JOSÉ DAVID MARTÍN
GUERRERO

YOLANDA VIVES
GILABERT



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

CLASIFICACIÓN DE EEG REALES BASADA EN TÉCNICAS CUÁNTICAS DE RESERVOIR COMPUTING.

AUTOR: MANUEL PÉREZ PERDOMO

JOSÉ DAVID MARTÍN

GUERRERO

TUTORES:

YOLANDA VIVES

GILABERT

Declaración de autoría:

Yo, Manuel Pérez Perdomo, declaro la autoría del Trabajo Fin de Grado titulado “Clasificación de EEG reales basada en técnicas cuánticas de reservoir computing.” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 28 de julio de 2025

Fdo: Manuel Pérez Perdomo

Resumen:

Este Trabajo de Fin de Grado (TFG) se encarga de proponer una versión cuántica de la técnica Reservoir Computing para la clasificación de electroencefalogramas (EEG) reales. Se empieza partiendo de una versión clásica de esta técnica para a continuación desarrollar un algoritmo cuántico que cumpla las mismas prestaciones y a poder ser, que consiga dar un mejor rendimiento. Para asegurar el correcto funcionamiento de ambas versiones, primero se realizarán pruebas con señales sintéticas unidimensionales, luego señales EEG simuladas y por último, señales EEG reales.

Abstract:

This Final Degree Project (TFG) proposes a quantum version of the Reservoir Computing technique for the classification of real electroencephalogram (EEG) signals. It starts from a classical version of this technique and then, I develop a quantum algorithm that meets the same performance and, if possible, that achieves better performance. To ensure the correct functioning of both versions, I will first test one-dimensional synthetic signals, then use simulated EEG signals and finally, real EEG signals.

Resum:

Aquest Treball de Fi de Grau (TFG) s'encarrega de proposar una versió quàntica de la tècnica Reservoir Computing per a la classificació de senyals d'electroencefalograma (EEG) reals. Parteix d'una versió clàssica d'esta tècnica per a després desenvolupar un algorisme quàntic que complisca les mateixes prestacions i a poder ser, que assolisca un millor rendiment. Per assegurar el correcte funcionament de les versions, primer es realitzaran proves amb senyals sintètics unidimensionals, després amb senyals EEG simulades i finalment, amb senyals EEG reals.

Agradecimientos:

En primer lugar, quiero agradecer a mi familia y amigos que han estado estos años conmigo y me apoyan cada día.

Seguidamente, quiero agradecer a todos mis amigos del grado por la ayuda y el cariño que he recibido por parte de ellos durante estos años.

Asimismo, agradecer a Jose y a Yolanda por la ayuda brindada durante la realización de este trabajo y por darme la oportunidad de poder integrarme al grupo de investigación QuIDAL durante estos meses. También, agradecer a todos los compañeros del grupo con los que he estado por su buen trato.

Muchas gracias a todos!

Índice general

1. Introducción	17
1.1. Motivación	17
1.2. Objetivos y estructura.	18
2. Marco Teórico	19
2.1. Estado del Arte	19
2.2. Reservoir Computing.	20
2.2.1. Concepto.	20
2.2.2. Funcionamiento.	20
2.3. Introducción a la Computación Cuántica.	22
2.3.1. Estados cuánticos, superposición y puertas cuánticas.	22
2.3.2. Quantum Machine Learning.	24
2.4. Reservoir Computing Cuántico.	25
2.5. Electroencefalografía.	26
3. Material y metodología	29
3.1. Datos del estudio.	29
3.2. Estructura del Reservoir Computing Clásico.	32
3.3. Estructura del Reservoir Computing Cuántico.	33
4. Aplicaciones y Resultados	35
4.1. RC en señal simple.	35
4.2. RC en señal compleja.	38
4.3. Clasificador de señales EEG.	41
4.3.1. Señales EEG simuladas.	41
4.3.2. Señales EEG reales.	47
5. Conclusiones y proyección futura.	51
5.1. Conclusiones	51
5.2. Trabajo futuro	52

Capítulo 1

Introducción

1.1. Motivación

La detección temprana y precisa de la actividad cerebral es fundamental en múltiples campos, como la *medicina*, la *neurociencia* y las *interfaces humano-máquina*. Las señales neurofisiológicas, en particular los *EEGs* (*electroencefalogramas*) [1], proporcionan una ventana para estudiar la dinámica cerebral de las personas. Sin embargo, la interpretación de estas señales es compleja, dado su volumen y su baja amplitud.

El uso de señales para la predicción de patrones cerebrales y actividades específicas se ha convertido en una herramienta crucial para mejorar diagnósticos, desarrollar tratamientos y potenciar tecnologías de asistencia. La capacidad de extraer información relevante de EEGs mediante técnicas de análisis y predicción puede marcar una diferencia significativa en la salud y calidad de vida de las personas, además de abrir nuevas fronteras en el entendimiento del cerebro humano.

Es por ello, que este trabajo se centra en el uso de estas señales y en la detección de distintos *estados cerebrales* a través de una técnica llamada *Reservoir Computing* (RC) [2]. Esto es un marco computacional adecuado para el procesamiento de datos temporales o secuenciales. Se deriva de varios modelos de redes neuronales recurrentes, incluidos los *echo state networks* [3] y *liquid state machines* [4]. Un sistema de Reservoir Computing consta de un reservorio, que proyecta las entradas a un espacio de alta dimensión, y un módulo de salida que analiza los patrones a partir de los estados en ese espacio. El reservorio se mantiene fijo, y solo se entrena el módulo de salida mediante un método simple, ya sea de regresión o clasificación.

A los sistemas de Reservoir Computing se les pueden dar distintos enfoques, siendo uno de ellos una versión *cuántica*. Los algoritmos cuánticos se caracterizan por utilizar como unidad de información el *qubit*; en lugar de trabajar únicamente con los estados 0 y 1, se emplean combinaciones probabilísticas de ambos, lo que ha impulsado su desarrollo y aplicación en diversos campos de la ciencia. Por tanto, una implementación cuántica de Reservoir Computing podría aprovechar las ventajas de la computación cuántica para descubrir relaciones y patrones que podrían superar a los obtenidos mediante su contraparte clásica.

En este trabajo, se va a desarrollar una técnica clásica y una cuántica de Reservoir Computing y en la capa final, haremos un *clustering* para la detección de los estados cerebrales.

1.2. Objetivos y estructura.

El objetivo principal de este trabajo es la implementación de manera clásica y cuántica de la técnica de Reservoir Computing, explorando diferentes enfoques y optimizaciones, y aplicarlo a señales EEG reales. A partir de este objetivo principal, se derivan varios objetivos:

- Conocer y comprender la técnica clásica de Reservoir Computing, sus fundamentos teóricos y su aplicación en el procesamiento de datos secuenciales.
- Estudiar los principios de los algoritmos cuánticos y explorar su aplicación en el desarrollo de una versión cuántica de Reservoir Computing, aprovechando las propiedades de la computación cuántica para mejorar el rendimiento del sistema.
- Familiarizarse con el uso de señales neurofisiológicas, en particular los *EEG*, y comprender la importancia de estas señales para la detección de la actividad cerebral.
- Desarrollar un clasificador a partir de la salida del Reservoir Computing, que permita distinguir entre la actividad cerebral de personas jóvenes y adultas, facilitando así una clasificación basada en edad.

La organización del trabajo es la siguiente:

- **Capítulo 2:** en este capítulo se habla del estado del arte del RC tanto de las primeras versiones clásicas como de las cuánticas. Posteriormente, se entra a definir y a explicar el funcionamiento de esta técnica. Como siguiente paso, se hace una introducción a la Computación Cuántica donde se habla de estados cuánticos, superposición y puertas cuánticas, y se hace una introducción al *Quantum Machine Learning*. Luego, entramos a definir el concepto de RC cuántico y por último, hacemos una sección acerca de los EEG.
- **Capítulo 3:** está dedicado a hablar de los datos aplicados durante el estudio y de las estructuras de nuestras versiones de RC clásico y cuántico.
- **Capítulo 4:** comentaremos las diversas aplicaciones que hemos hecho y pondremos los resultados obtenidos para cada una de las pruebas realizadas y los resultados finales.
- **Capítulo 5:** desarrollamos las conclusiones y la proyección futura de este trabajo.

El código del RC clásico y cuántico, así como todas las pruebas y resultados de este trabajo, se pueden consultar en el repositorio de GitHub del proyecto. Para entender la organización de los notebooks, se recomienda leer el archivo `README.md`¹.

¹<https://github.com/manuuuv/TFG/tree/TFG>

Capítulo 2

Marco Teórico

2.1. Estado del Arte

En términos generales, el concepto de Reservoir Computing surge del uso de conexiones recursivas dentro de las redes neuronales, con el fin de generar un sistema dinámico complejo [5]. La complejidad resultante de dichas redes neuronales recurrentes demostró ser útil para resolver una variedad de problemas, en campos como el procesamiento del lenguaje y en la modelación de sistemas dinámicos. Sin embargo, el entrenamiento de redes neuronales recurrentes resulta complicado y computacionalmente costoso. El Reservoir Computing mitiga estos problemas relacionados con el entrenamiento al fijar la dinámica del reservoir y limitar el entrenamiento únicamente a la capa de salida lineal.

En general, el concepto general de Reservoir Computing surge del uso de conexiones recursivas dentro de las redes neuronales para generar un sistema dinámico complejo. Se trata de una generalización de arquitecturas anteriores como las redes neuronales recurrentes, las *liquid-state machines* y las *echo-state networks*. El enfoque de Reservoir Computing también se extiende a sistemas físicos que no son redes en el sentido clásico, sino sistemas continuos en el espacio y/o el tiempo; por ejemplo, un simple “recipiente con agua” puede funcionar como un reservorio que realiza cálculos a partir de perturbaciones en su superficie.

Los primeros ejemplos de redes neuronales de tipo reservoir demostraron que las redes neuronales recurrentes con conexiones aleatorias podían utilizarse para el aprendizaje de secuencias sensoriomotoras [6], así como para formas simples de discriminación de intervalos y de habla [7]. En estos primeros modelos [8], la memoria en la red se manifestaba tanto a través de la plasticidad sináptica a corto plazo como mediante la actividad generada por las conexiones recurrentes [9]. En otros modelos iniciales de redes *reservoir*, la memoria del historial reciente del estímulo era proporcionada exclusivamente por la actividad recurrente.

Avances recientes tanto en Inteligencia Artificial como en Teoría de la Información Cuántica han dado lugar al concepto de redes neuronales cuánticas [10]. Estas presentan un gran potencial para el procesamiento de información cuántica — una tarea difícil para las redes clásicas —, aunque también pueden aplicarse a la resolución de problemas clásicos.

En 2018, se logró una arquitectura de Reservoir Computing cuántico, utilizando espines nucleares dentro de un sólido molecular [11]. Sin embargo, los experimentos con

espines nucleares mencionados no demostraron computación de reservorio cuántica como tal, ya que no implicaron el procesamiento de datos secuenciales. En su lugar, se utilizaron entradas vectoriales, lo que convierte este enfoque, más precisamente, en una demostración de una implementación cuántica del algoritmo conocido como *random kitchen sink* (también denominado *extreme learning machines* en algunas comunidades).

En 2019, se propuso otra posible implementación de procesadores cuánticos de reservorio basada en redes fermiónicas bidimensionales. Posteriormente, en 2020, se planteó y demostró la realización de Reservoir Computing en ordenadores cuánticos digitales, utilizando los sistemas superconductores cuánticos en la nube de IBM [12].

2.2. Reservoir Computing.

2.2.1. Concepto.

La técnica Reservoir Computing abrió un nuevo paradigma en el entrenamiento de las redes neuronales. La idea central del RC [13] es que una parte significativa de la tarea informática no la realiza una red entrenada, sino un sistema de muy alta dimensión (reservorio) que es capaz de realizarla por sí mismo, y cuya salida se introduce en una única capa de salida, que es el único componente de la red que se entrena.

Técnicamente, el marco de Reservoir Computing se basa en una arquitectura de red neuronal recurrente (*RNN*) [14], donde una red recurrente fija y aleatoriamente conectada de neuronas artificiales (el reservorio) transforma las señales de entrada en un espacio de alta dimensión.

La presencia de conexiones de retroalimentación o recurrentes es un aspecto clave para el tratamiento de información temporal, lo que hace que el RC sea adecuado para tareas que implican patrones temporales, como la predicción de series, el reconocimiento del habla y la música.

En RC, las señales de entrada se introducen en el reservorio, donde generan patrones dinámicos de actividad. A continuación, se entrena un modelo, ya sea de regresión, clasificación o clustering, utilizando las salidas del reservorio para producir los resultados deseados.

En comparación, en un sistema clásico típico de *Deep Learning*, todas las capas de la red deben entrenarse mediante retropropagación. Este proceso es intensivo y poco eficiente para el tratamiento de datos secuenciales. RC representa una alternativa más económica al Deep Learning clásico, ya que solo requiere el entrenamiento de la capa de salida.

En la figura 2.1 se muestra una ilustración de la estructura del Reservoir Computing de manera básica.

2.2.2. Funcionamiento.

El principio general del RC resulta intuitivo [15]. Permite construir un sistema dinámico capaz de implementar un mapeo deseado desde señales de entrada a señales de salida. Esto incluye tareas como la predicción de series temporales, clasificación, entre otras.

En este esquema, se proporciona una señal de entrada de entrenamiento junto con una señal de salida objetivo deseada. En predicción de series temporales, la señal de entrada

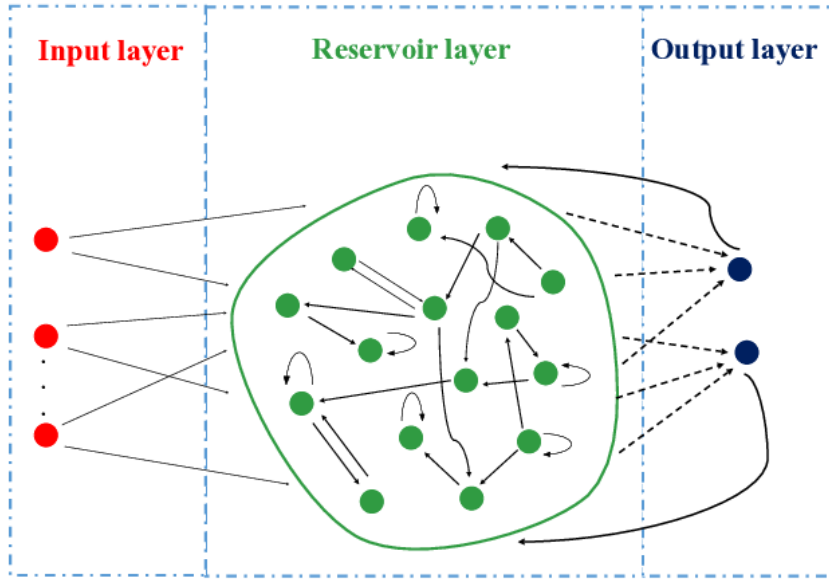


Figura 2.1: Estructura del Reservoir Computing. Se puede observar que tenemos tres capas: capa de entrada, capa del reservorio y capa de salida [13].

puede representar una serie temporal y la señal de salida objetivo su continuación. En clasificación, la señal de entrada puede representar una señal de voz auditiva y la salida objetivo los fonemas correspondientes. Cabe destacar que si existen múltiples ejemplos de entrada-salida, estos pueden concatenarse en el tiempo. Lo que se desea es un sistema que, al recibir como entrada $u_{\text{train}}(t)$, genere una señal de salida correspondiente. En este trabajo, cambia un poco este panorama, ya que tenemos la señal de entrada, pero no tenemos una señal objetivo a la que nos queramos parecer, sino que usando la señal de entrada hacemos un *clustering* de los valores de la salida del reservorio lo que se corresponde con un aprendizaje no supervisado. Sin embargo, vamos a seguir describiendo estos ejemplos que nos van a ayudar a comprender el funcionamiento general del RC.

Podemos asumir que los ejemplos anteriores se generan mediante una función $\Psi(u_{\text{train}})(t) = y_{\text{train}}(t) + e$, donde e es un término de error aditivo, por ejemplo, ruido, y u_{train} es la serie temporal completa de entrada. En procesamiento de señales, a un sistema de este tipo se le denomina *filtro*. El sistema de reservorio, denotado por $\mathcal{R}_\theta$ (donde θ es el vector de parámetros del modelo que determina el sistema), tiene como objetivo aproximar lo mejor posible al sistema generador verdadero Ψ .

La solución que ofrece Reservoir Computing a esta tarea de aprendizaje se divide en tres pasos:

(a) El reservorio se prepara como un sistema dinámico no lineal de alta dimensión que puede ser impulsado por la señal de entrada $u_{\text{train}}(t)$. Los componentes individuales del reservorio pueden ser aleatorios. Los estados del reservorio se denotan por $\mathbf{x}(t)$, y muchas de sus variables de estado deben ser accesibles para ser leídas como $\mathbf{x}_i(t)$ con $i = 1, \dots, N_x$. No es necesario que todos los estados del reservorio sean observables; es posible que $N_x < N$.

(b) El reservorio se alimenta muestra a muestra con la entrada $u_{\text{train}}(t)$, y las variables de estado correspondientes del reservorio, como respuesta a esta entrada, se registran y almacenan como $\mathbf{x}_i(t)$.

(c) Finalmente, se aprende (estima) una función de salida F , que mapea todos los

vectores de estado registrados $\mathbf{x}(t)$ a una salida $\hat{y}(t)$. Esto se realiza minimizando una función de pérdida $\mathcal{L}$ elegida, que evalúa la diferencia entre la salida del reservorio $\hat{y}(t)$ y la salida deseada $y = \Psi(u)$. Específicamente, se minimiza la pérdida media a lo largo de los datos de entrenamiento:

$$\frac{1}{N_T} \sum_{t=1}^{N_T} \mathcal{L}(\hat{y}(t), y(t)) \quad \text{donde } N_T \text{ es el número de pasos temporales. La función de salida } F,$$

que minimiza esta pérdida promedio.

Después de este procedimiento, el reservorio puede utilizarse para inferencia o predicción. Al introducir una nueva señal de entrada $u(t)$, se observan los vectores de estado $\mathbf{x}_i(t)$ del reservorio y se utilizan para calcular la señal de salida predicha $\hat{y}(t)$.

2.3. Introducción a la Computación Cuántica.

La *computación cuántica* y la *información cuántica* son el estudio de las tareas de procesamiento de la información que pueden realizarse utilizando sistemas de *mecánica cuántica*. Esta se puede definir como un marco matemático o conjunto de reglas para la construcción de teorías físicas. Una de las ventajas de la computación cuántica es que promete resolver problemas que son irresolubles en los ordenadores digitales. Los algoritmos cuánticos altamente paralelos pueden disminuir el tiempo de cómputo para algunos problemas en muchos órdenes de magnitud.

En este trabajo, no nos vamos a centrar mucho en estos términos, pero sí que vamos a desarrollar una serie de conceptos que son importantes saber para entender la técnica cuántica de Reservoir Computing. Si se desea obtener más información acerca de estos temas, pueden consultar [16] y [17].

2.3.1. Estados cuánticos, superposición y puertas cuánticas.

A continuación, redactaremos las principales diferencias entre el bit clásico y el bit cuántico, también conocido como *qubit*, y con respecto a este, veremos el fenómeno de la superposición cuántica [18]. Asimismo, desarrollaremos el término de puertas cuánticas, ya que serán utilizadas posteriormente.

Los bits clásicos, las unidades básicas de información en la computación clásica, pueden existir en uno de dos estados distintos, típicamente representados como 0 o 1. Esta representación binaria es la piedra angular de la computación clásica, donde todos los datos, instrucciones y operaciones se reducen en última instancia a secuencias de 0 y 1. La principal característica de estos es que son deterministas; en un momento dado, un bit se encuentra inequívocamente en un estado u otro. Esta clara distinción permite el almacenamiento y manipulación confiable de información utilizando puertas lógicas clásicas (por ejemplo, AND, OR, NOT), que realizan operaciones en uno o más bits para producir una salida definida.

Por el contrario, los *qubits* aprovechan los principios de la mecánica cuántica, en particular la *superposición* y el *entrelazamiento*, para representar y procesar información de maneras que los bits clásicos no pueden. Un *qubit* puede existir no sólo en los estados

clásicos $|0\rangle$ o $|1\rangle$, sino también en cualquier superposición cuántica de estos estados. Matemáticamente, esto se expresa como:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad \text{donde } |\psi\rangle \text{ es el estado del qubit, y } \alpha \text{ y } \beta \text{ son números complejos que satisfacen la condición de normalización } |\alpha|^2 + |\beta|^2 = 1.$$

Los coeficientes α y β representan amplitudes de probabilidad, y sus magnitudes al cuadrado dan las probabilidades de que el qubit se mida en el estado $|0\rangle$ o $|1\rangle$, respectivamente.

La superposición permite que un qubit codifique más información que un bit clásico. Mientras que un bit clásico se limita a representar un único valor binario en cualquier momento, un qubit en superposición puede representar simultáneamente 0 y 1, aunque de forma probabilística. Esta propiedad permite a los ordenadores cuánticos procesar una gran cantidad de información en paralelo, aumentando exponencialmente su poder computacional para determinadas tareas.

Otra distinción crítica es el *entrelazamiento*, un fenómeno exclusivamente cuántico en el que los estados de dos o más qubits se correlacionan de tal manera que el estado de un qubit no puede describirse independientemente del estado de los demás e independientemente de la distancia que los separe. Los qubits entrelazados exhiben correlaciones que son más fuertes que cualquier contraparte clásica, lo que permite a los ordenadores cuánticos realizar operaciones complejas de manera más eficiente. Por ejemplo, en un sistema de qubits entrelazados, una medición en un qubit afecta instantáneamente el estado de su compañero entrelazado.

Las capacidades de procesamiento de los qubits se ven reforzadas aún más por las *puertas cuánticas* que, a diferencia de las puertas lógicas clásicas, operan según los principios de transformaciones unitarias.

Las puertas cuánticas manipulan los qubits mediante operaciones que preservan la probabilidad total (es decir, son reversibles). Las puertas cuánticas comunes incluyen la puerta Pauli-X (análoga a la puerta NOT clásica) y cuya representación matricial es la siguiente:

$$\begin{aligned} X &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \\ Y &= \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \\ Z &= \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}. \end{aligned}$$

La puerta Hadamard que crea superposiciones transformando un qubit $|0\rangle$ en $\frac{|0\rangle+|1\rangle}{\sqrt{2}}$ y $|1\rangle$ en $\frac{|0\rangle-|1\rangle}{\sqrt{2}}$, representada por la matriz:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

La puerta Control-NOT (CNOT) entrelaza dos qubits, invirtiendo el qubit objetivo cuando el qubit de control está en $|1\rangle$, siendo una operación clave para crear estados

entrelazados:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Estas puertas se pueden combinar para formar *circuitos cuánticos*, que realizan cálculos aprovechando la superposición, el entrelazamiento y la interferencia.

2.3.2. Quantum Machine Learning.

Al ya conocer un poco los fundamentos más relevantes de la computación cuántica, podemos avanzar un poco y entrar en el tema del *Quantum Machine Learning* [19].

Sin embargo, antes de ello, vamos a hacer una breve introducción al concepto de *Machine Learning* (ML). Este es un subconjunto de la Inteligencia Artificial (IA) que se centra en el desarrollo de algoritmos informáticos que mejoran automáticamente mediante la experiencia y el uso de datos. En términos más sencillos, el Machine Learning permite a los ordenadores aprender de los datos y tomar decisiones o hacer predicciones sin estar explícitamente programados para ello.

Una vez explicado esto, podemos definir el *Quantum Machine Learning* (QML) como un campo emergente que aprovecha los principios de la mecánica cuántica para mejorar el entrenamiento, rendimiento y escalabilidad de los modelos de Machine Learning. La computación cuántica ofrece una solución transformadora a los desafíos computacionales del ML, al procesar de manera eficiente datos de alta dimensionalidad y resolver problemas de optimización complejos. QML integra la computación cuántica con el ML para resolver problemas que van más allá del alcance de los métodos clásicos.

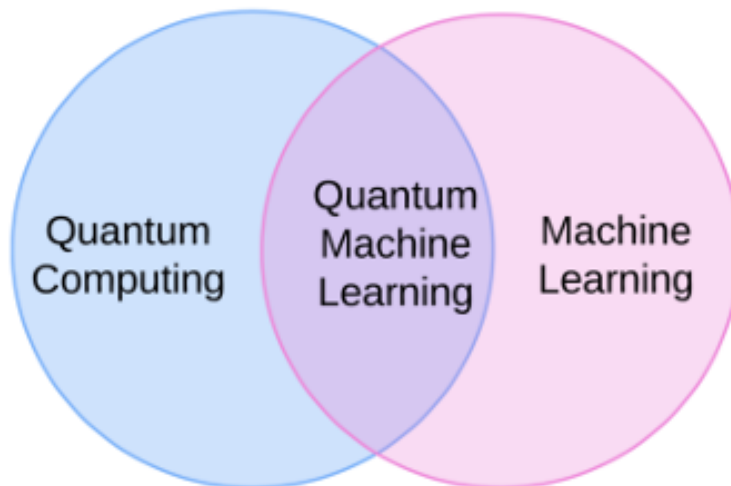


Figura 2.2: Quantum Machine Learning: Intersección de Quantum Computing y Machine Learning.¹

Los algoritmos de Quantum Machine Learning se desarrollan en simuladores cuánticos como *IBM's Qiskit* [20], *Google's Cirq* [21], *TensorFlow Quantum* [22] y *PennyLane* [23]. Recientemente, se introdujo *sQULearn* [24], una librería de Python preparada para aprendizaje cuántico que se integra perfectamente con *scikit-learn*, proporcionando interfaces

¹Imagen extraída de https://cdn-images-1.medium.com/max/800/1*LmUyhII4sv2Sn3R2tPt8Dw.png

de alto nivel para redes neuronales cuánticas (*QNNs*) y kernels cuánticos. Esta librería permite a investigadores y practicantes desarrollar y ejecutar modelos de QML de manera eficiente utilizando *Qiskit* y *PennyLane* como backends.

En este trabajo, para el desarrollo del algoritmo cuántico se va a utilizar el simulador *PennyLane* desde el lenguaje Python.

2.4. Reservoir Computing Cuántico.

Una vez vistos los fundamentos principales del Quantum Computing y el Quantum Machine Learning, ahora vamos a ver el concepto de *Reservoir Computing Cuántico* (QRC) [25].

Este surge de la unión entre Quantum Computing y Reservoir Computing, con el objetivo de aprovechar la dinámica compleja de los sistemas cuánticos como un reservorio para tareas de aprendizaje automático en tiempo real.

QRC incorpora al paradigma de los reservorios características contraintuitivas propias de la física cuántica, como la superposición y el entrelazamiento. Estos aspectos fundamentales han sido discutidos previamente en el apartado 2.3.1, donde se presentan sus definiciones y propiedades más relevantes.

Una de las principales fortalezas del *Quantum Reservoir Computing* es precisamente la capacidad de potenciar y explotar estas propiedades cuánticas para abordar problemas computacionales complejos que, en un contexto clásico, resultarían costosos en términos de tiempo o recursos. En particular, el entrelazamiento cuántico juega un papel esencial, ya que permite crear correlaciones no locales entre los estados de diferentes qubits, generando un espacio de estados mucho más amplio. Estas correlaciones permiten representar y procesar información de forma distribuida, lo que aumenta el poder de representación del sistema.

En el marco del QRC, este entrelazamiento suele generarse mediante puertas cuánticas como la CNOT (Controlled-NOT), que introduce correlaciones entre pares de qubits al actuar de forma condicional: el segundo qubit (target) cambia su estado si el primero (control) está en estado $|1\rangle$. Este tipo de operación es clave para crear entrelazamiento entre qubits, y su correcta implementación dentro del reservorio permite capturar dinámicas altamente no lineales en el procesamiento de información.

Junto con la CNOT, otra puerta fundamental es la RX, una rotación unitaria en torno al eje X . Esta puerta actúa localmente sobre un qubit, modificando su fase y amplitud de probabilidad, y resulta especialmente útil para explorar diferentes configuraciones del espacio de estados del sistema.

En la figura 2.3 se puede observar la estructura general de un *Reservoir Computing Cuántico*, en la que estas operaciones cuánticas se integran como parte esencial de la dinámica interna del sistema.

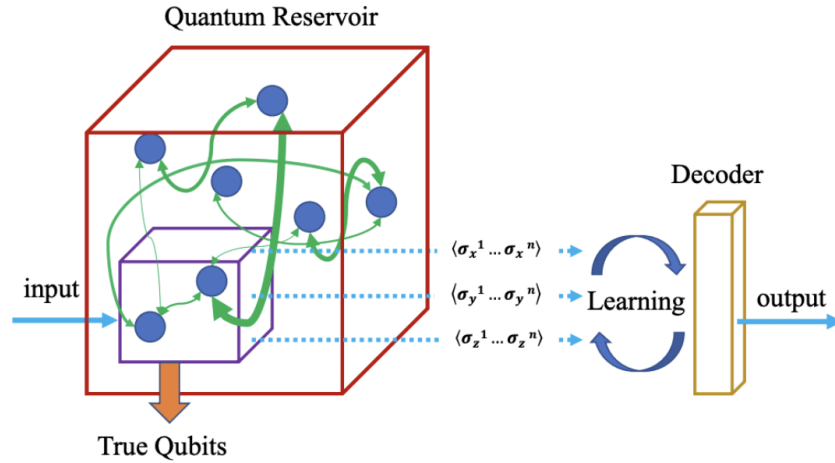


Figura 2.3: Quantum Reservoir Computing: Arquitectura y capas del Quantum Reservoir.²

2.5. Electroencefalografía.

Tal y como se indica en el apartado 1.1, en este proyecto vamos a trabajar con señales EEGs (electroencefalogramas) por lo que vamos a definir el concepto de estos y sus principales características.

Un electroencefalograma [1] es una prueba que mide la actividad eléctrica del cerebro. Esta prueba requiere la colocación en el cuero cabelludo de electrodos, que son unos pequeños discos metálicos. Las neuronas cerebrales se comunican mediante impulsos eléctricos, y esta actividad se manifiesta como líneas onduladas en un registro electroencefalográfico. El electroencefalograma puede detectar cambios en la actividad cerebral que podrían ayudar a diagnosticar afecciones cerebrales, especialmente epilepsia u otras afecciones convulsivas.

Un término importante de los EEG son los *canales*, que se refieren a las ubicaciones específicas donde se colocan los electrodos. Cada canal captura la actividad eléctrica de una región cerebral específica. Las características de los canales EEG incluyen su amplitud (voltaje), frecuencia (ritmo) y forma de onda, que varían dependiendo del estado del cerebro y la actividad neuronal subyacente. Entre las formas de onda más comunes se encuentran:

- **Delta** (< 4 Hz): asociada con el sueño profundo y estados inconscientes.
- **Theta** (4–7 Hz): vinculada con la somnolencia, meditación profunda o estados de ensoñación.
- **Alfa** (8–13 Hz): relacionada con la relajación en estado de vigilia, especialmente con los ojos cerrados.
- **Beta** (13–30 Hz): típica de estados de alerta, concentración y actividad mental intensa.

²Imagen extraída de <https://arxiv.org/html/2403.08998v2/x1.png>

La tabla 2.1 describe las características principales de los canales propios de los EEG:

Tabla 2.1: Características principales de los canales del EEG

Canales	Ubicación General	Descripción Funcional
Fp1, Fp2, Fpz	Prefrontal	Procesos ejecutivos, atención, control cognitivo.
AF7, AF3, AF4, AF8, AFz	Prefrontal anterior	Integración sensorial, toma de decisiones.
F1, F2, F3, F4, F5, F6, F7, F8, Fz	Frontal	Planificación motora, funciones cognitivas.
FT7, FT8	Frontotemporal	Procesamiento auditivo, lenguaje.
FC1, FC2, FC3, FC4, FC5, FC6, FCz	Frontocentral	Activación motora, áreas premotoras.
C1, C2, C3, C4, C5, C6, Cz	Central	Corteza motora primaria, control de movimientos.
T7, T8	Temporal lateral	Procesamiento auditivo, percepción del habla.
TP7, TP8	Temporoparietal	Integración multisensorial, lenguaje.
CP1, CP2, CP3, CP4, CP5, CP6, CPz	Centro-parietal	Procesamiento somatosensorial (tacto, temperatura).
P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, Pz	Parietal	Atención, percepción espacial y táctil.
PO3, PO4, PO7, PO8, POz	Parieto-occipital	Integración visual y espacial.
O1, O2, Oz	Occipital	Procesamiento visual primario.
Iz	Inion (zona occipital inferior)	Punto de referencia, útil para orientación espacial.

Para un mejor entendimiento de la ubicación de los canales en el cerebro, se puede ver en la figura 2.4:

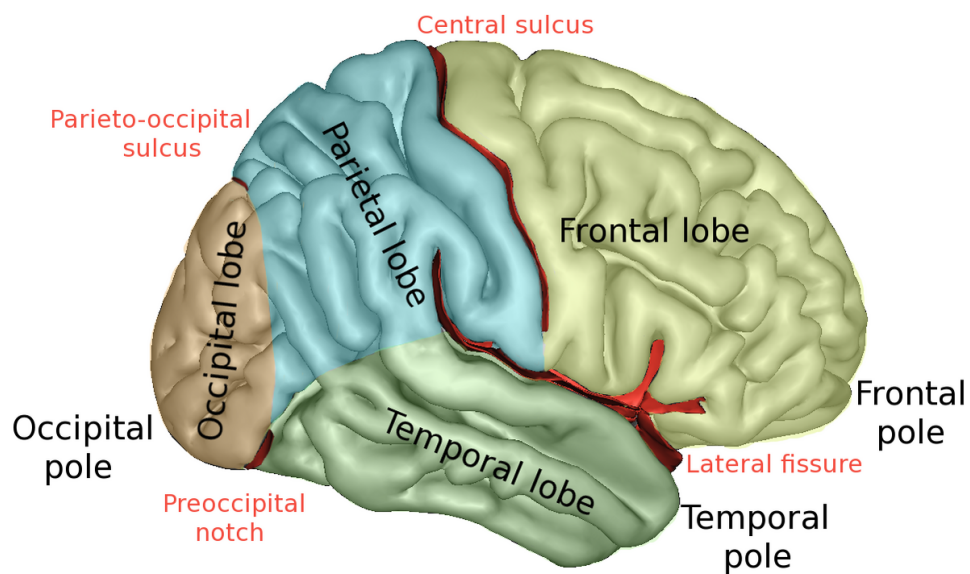


Figura 2.4: Mapa lobular del cerebro: frontal, frontotemporal, parietal, temporo-lateral y occipital.³

³Imagen extraída de <https://upload.wikimedia.org/wikipedia/commons/thumb/4/46/LobesCapsLateral.png/1200px-LobesCapsLateral.png>

Capítulo 3

Material y metodología

En el capítulo anterior, se ha explicado el concepto de RC y su relación con la computación cuántica, además de definir qué son los EEG y la importancia que tienen en nuestro trabajo. Por tanto, una vez hecho esto, este capítulo se centrará en las contribuciones propias del TFG. Comenzaremos desarrollando los datos utilizados y definiremos la estructura de nuestro RC clásico y cuántico.

3.1. Datos del estudio.

Como previamente se ha comentado, en este trabajo usaremos señales EEG tomadas a personas reales. Las señales serán divididas en dos grupos: EEG de personas adultas jóvenes y de personas adultas mayores en estado de reposo. Atendiendo a esta división en grupos, tenemos 23 señales de personas jóvenes y 24 de personas mayores. Estas señales han sido proporcionadas por investigadores de la *Universidad de Valparaíso, Chile* con los cuales colabora el grupo de investigación IDAL, de la ETSE-UV.

Al igual que se ha comentado en el apartado 2.5, cada señal tiene 64 canales, lo que las hace tener una alta dimensión. A estas señales se les ha hecho previamente un *downsampling* para reducir la carga computacional y un análisis de componentes independientes (ICA) para eliminar artefactos. Cabe destacar que la frecuencia de muestreo de las señales es de 1024 Hz. En la figura 3.1 se puede observar una muestra del primer canal de las señales en un sujeto joven y adulto.

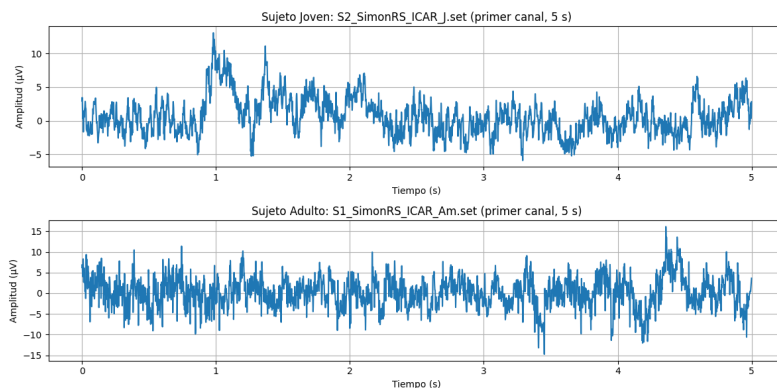


Figura 3.1: Muestra del primer canal EEG de sujeto Joven (arriba) y Adulto (abajo).

Sin embargo, antes de integrar las señales EEG a nuestro RC, haremos unas pruebas iniciales con distintas señales en las que la dificultad de estas irá aumentando progresivamente.

En primer lugar, se realizó una prueba inicial utilizando una señal sintética unidimensional. Esta señal tiene una duración total de 6 segundos y fue generada con una frecuencia de muestreo de 55 Hz. Se compone de seis segmentos consecutivos de un segundo cada uno, en los que se alternan dos oscilaciones senoidales con frecuencias distintas: 3.5 Hz y 11.5 Hz. De este modo, los segmentos con índice par contienen una oscilación de 3.5 Hz, mientras que los impares contienen una de 11.5 Hz.

La señal resultante se construyó concatenando estas oscilaciones de forma alternada, manteniendo una amplitud constante en todos los segmentos. Se puede observar en la figura 3.2 el resultado de esta:

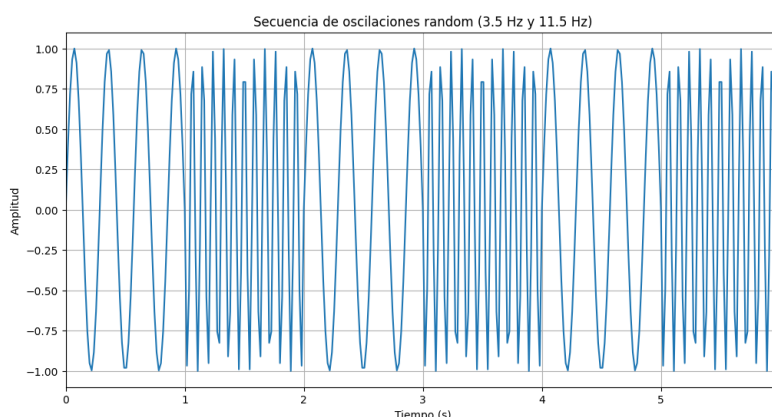


Figura 3.2: Señal sintética unidimensional compuesta por seis segmentos alternados de 3.5 Hz y 11.5 Hz. Esta señal fue utilizada como prueba inicial para observar el comportamiento del sistema en un entorno controlado.

A continuación, se diseñó una señal sintética más compleja. Esta nueva señal también es unidimensional y fue generada con una frecuencia de muestreo de 55 Hz y una duración total de 32 segundos.

La señal se construyó a partir de la concatenación de oscilaciones senoidales con cuatro frecuencias distintas: 3.5 Hz, 11.5 Hz, 7.0 Hz y 15.0 Hz. Cada una de estas oscilaciones ocupa un segmento de 4 segundos de duración, y el ciclo completo de cuatro frecuencias se repite dos veces, generando así un total de ocho segmentos consecutivos.

Esta configuración permite observar el comportamiento del RC ante una señal más rica en contenido frecuencial y con transiciones más variadas. En la figura 3.3 se encuentra el resultado de esta:

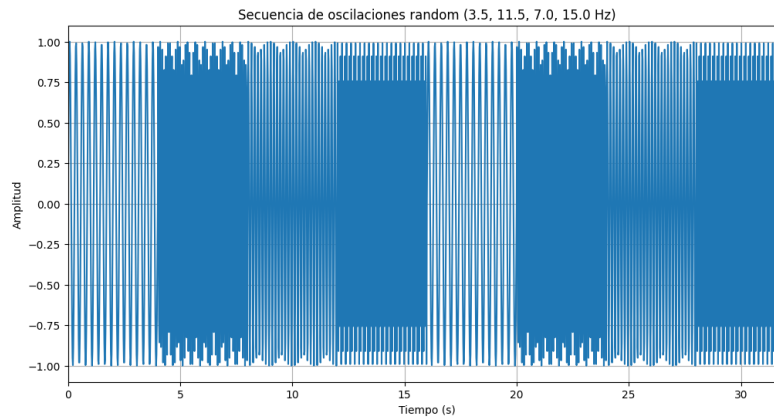


Figura 3.3: Señal sintética más compleja compuesta por ocho segmentos senoidales que alternan entre las frecuencias 3.5 Hz, 11.5 Hz, 7.0 Hz y 15.0 Hz. Esta señal simula una dinámica más variada y sirve como paso intermedio hacia el análisis de señales EEG reales.

Como última prueba, hemos utilizado unas señales EEG que han sido generadas sintéticamente. Estas presentan características controladas para cada grupo (jóvenes y mayores) y permiten simular distintas dinámicas propias de la actividad cerebral sin depender todavía de datos reales, facilitando así una evaluación más controlada del comportamiento del sistema. De estas señales tenemos 40 en total, siendo 20 de sujetos jóvenes y las 20 restantes de mayores.

La generación de estas señales se realizó teniendo en cuenta cuatro bandas de frecuencia de EEG: Delta (0.5–4 Hz), Theta (4–8 Hz), Alpha (8–13 Hz) y Beta (13–30 Hz). Las señales de los sujetos jóvenes se caracterizan por una mayor actividad en la banda Beta, donde se introdujo un pico de amplitud en dicha banda, mientras que las señales de los sujetos mayores presentan menor amplitud y un mayor contenido de ruido.

Además, para aumentar el realismo de las señales, se añadió componentes generadas mediante procesos autorregresivos y procesos gaussianos, los cuales imitan las propiedades estadísticas de señales EEG reales. Cada señal contiene múltiples canales, 10 en concreto, y una alta frecuencia de muestreo (512 Hz). Este conjunto de datos sintéticos constituye un paso intermedio útil para validar la capacidad del sistema antes de introducir las señales EEG reales. Se puede observar en la figura 3.4 una muestra de estas señales sintéticas para un sujeto joven y mayor.¹

¹Los datos sintéticos han sido generados a partir del cuaderno `synthetic_eeg_v10.ipynb`, disponible en el repositorio github.com/jogugil/MyRC. Los datos pueden encontrarse en la carpeta `synthetic_data` bajo el nombre `synthetic_data_auto.npy`.

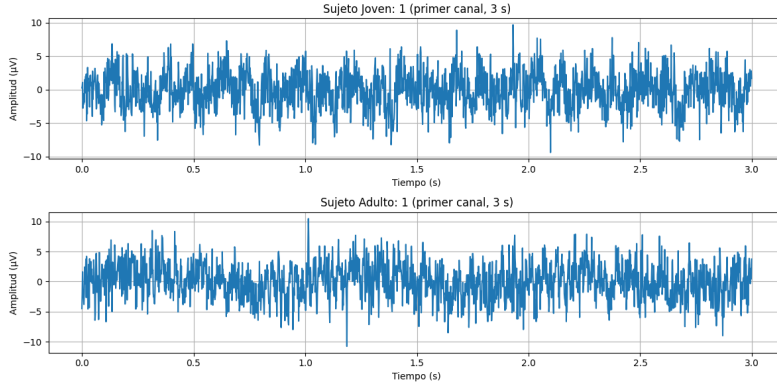


Figura 3.4: Muestra del primer canal de señal sintética de sujeto Joven (arriba) y Adulto (abajo).

3.2. Estructura del Reservoir Computing Clásico.

A continuación, vamos a definir la estructura de nuestro RC clásico. Antes de ello, hay que destacar que esta estructura se basa en una comunicación en el congreso OHBM [26] el cual también trabaja con señales EEG y que tiene como objetivo la identificación de distintos estados cerebrales usando la técnica de Reservoir Computing, al igual que nosotros. Asimismo, cabe decir que la estructura del RC es parecida, pero cambia en varios aspectos que serán comentados más adelante.

En primer lugar, nuestro RC clásico ha sido implementado usando el lenguaje de programación Python. El modelo se encuentra compuesto por tres capas: capa de entrada, el reservorio y la capa de salida.

Teniendo en cuenta que estamos en la versión más simple, se considera una señal unidimensional de entrada ($n_{\text{in}} = 1$), un número total de neuronas en el reservorio denotado por N , y una única neurona de salida ($n_{\text{out}} = 1$). La longitud total de la secuencia de entrada de entrenamiento es T .

Los pesos que conectan la entrada con el reservorio se inicializan aleatoriamente a partir de una distribución normal estándar:

$$\mathbf{W}_{\text{in}} \in \mathbb{R}^{N \times n_{\text{in}}} \sim \mathcal{N}(0, 1).$$

De igual forma, los pesos internos del reservorio se generan aleatoriamente:

$$\mathbf{W}_{\text{RC}} \in \mathbb{R}^{N \times N} \sim \mathcal{N}(0, 1),$$

y posteriormente se normalizan columna por columna para controlar la dinámica interna:

$$\mathbf{W}_{\text{RC}} \leftarrow \frac{\mathbf{W}_{\text{RC}}}{\|\mathbf{W}_{\text{RC}}\|_{\text{col}}}.$$

Adicionalmente, se incorpora un vector de sesgo para las neuronas del reservorio, generado a partir de una distribución uniforme:

$$\mathbf{b}_{\text{in}} \in \mathbb{R}^{N \times 1} \sim \mathcal{U}(0, 1).$$

La dinámica del sistema se describe mediante una evolución temporal del estado interno del reservorio. En cada instante de tiempo t , el estado del reservorio $\mathbf{x}_t \in \mathbb{R}^N$ se actualiza según la siguiente ecuación:

$\mathbf{x}_t = \tanh(\alpha \mathbf{W}_{\text{in}} \mathbf{u}_t + \beta \mathbf{W}_{\text{RC}} \mathbf{x}_{t-1} + \mathbf{b}_{\text{in}})$, donde $\mathbf{u}_t$ es el vector de entrada en el instante t , y α, β son factores de escalado que regulan la influencia de la entrada y del estado anterior, respectivamente (en este caso, $\alpha = 0,05$, $\beta = 0,95$).

El estado inicial $\mathbf{x}_0$ se inicializa de forma aleatoria:

$$\mathbf{x}_0 \sim \mathcal{N}(0, 1).$$

Una vez ejecutada la evolución dinámica a lo largo de todas las muestras de entrada, se construye una matriz $\mathbf{X} \in \mathbb{R}^{N \times (T+1)}$ que contiene los estados del reservorio a lo largo del tiempo. Esta matriz se centra respecto a la media de cada fila para eliminar sesgos estacionarios:

$$\tilde{\mathbf{X}} = \mathbf{X} - \text{media}(\mathbf{X}, \text{eje} = 1).$$

Para capturar la envolvente temporal de la actividad del reservorio, se aplica la transformada de Hilbert sobre la matriz centrada. Esta operación genera una señal analítica compleja, cuya magnitud representa la envolvente de amplitud del reservorio:

$\mathbf{A} = |\mathcal{H}(\tilde{\mathbf{Y}})|$, donde $\mathcal{H}$ denota la transformada de Hilbert, definida como

$$\mathcal{H}\{x(t)\} = \frac{1}{\pi} \text{p.v.} \int_{-\infty}^{\infty} \frac{x(\tau)}{t - \tau} d\tau,$$

y el valor absoluto representa la magnitud de la señal analítica.

La matriz $\mathbf{A}$ se utiliza como entrada a la capa de salida, donde aquí se aplicará una técnica de clustering para agrupar las muestras en distintos estados. En esta parte, sí que hay un entrenamiento de los pesos.

3.3. Estructura del Reservoir Computing Cuántico.

Una vez vista la estructura del RC clásico, ahora desarrollaremos nuestra versión de RC cuántico. Esta se caracteriza por tener implementado un circuito cuántico parametrizado. Este enfoque se ha desarrollado utilizando la librería **PennyLane**, un framework de computación cuántica híbrida compatible con interfaces clásicas en Python.

Codificación y evolución cuántica.

El sistema está compuesto por Q qubits, donde Q representa un número que determina la dimensión del reservorio cuántico. En cada paso temporal t , la entrada u_t y la salida previa del sistema $\mathbf{y}_{t-1}$ se codifican en los qubits mediante rotaciones controladas de la forma:

$$\text{RX} \left(\alpha \cdot w_i^{\text{in}} \cdot u_t + \beta \cdot w_i^{\text{rc}} \cdot y_{t-1}^{(i)} \right), \quad \text{para } i = 1, \dots, Q,$$

donde α y β son coeficientes de ponderación que regulan la influencia de la entrada y la memoria interna del sistema, y los pesos w_i^{in} , w_i^{rc} se inicializan aleatoriamente.

Tras la codificación, el estado cuántico evoluciona mediante un circuito con puertas de entrelazamiento del tipo CNOT, nombradas en la sección 2.3.1, entre qubits consecutivos (en anillo), intercaladas con rotaciones adicionales dependientes de una matriz de pesos cuánticos $\mathbf{W}_{\text{quant}} \in \mathbb{R}^{Q \times Q}$:

$$\text{CNOT}_{i,(i+1) \bmod Q}, \quad \text{RX}(\theta_{ij}).$$

Lectura y procesamiento de la señal cuántica.

Finalizada la evolución cuántica, se obtiene el vector de probabilidades $\mathbf{p}_t \in \mathbb{R}^{2^Q}$ asociado a los estados posibles de los qubits. Para mantener una dimensión constante del espacio latente, se aplica una función de activación tipo tangente hiperbólica:

$$\mathbf{y}_t = \tanh(\mathbf{p}_t),$$

obteniendo así un vector de tamaño Q que representa la salida del sistema en el instante t .

Estos vectores se concatenan en el tiempo para formar la matriz de salida del sistema $\mathbf{Y} \in \mathbb{R}^{T \times Q}$, la cual se centra respecto a la media de cada fila:

$$\tilde{\mathbf{Y}} = \mathbf{Y} - \text{media}(\mathbf{Y}, \text{eje} = 1).$$

Sobre esta matriz centrada se aplica la transformada de Hilbert para obtener la envolvente compleja de cada qubit a lo largo del tiempo:

$$\mathbf{A}_{\text{quant}} = |\mathcal{H}(\tilde{\mathbf{Y}})|, \quad \text{donde } \mathcal{H} \text{ denota la transformada de Hilbert y el valor absoluto representa la magnitud de la señal analítica.}$$

Capa de salida y análisis de estados.

La matriz $\mathbf{A}_{\text{quant}}$ se interpreta como una representación de la dinámica del sistema cuántico. Al igual que en el modelo clásico, se va a emplear una técnica de *clustering* no supervisada para agrupar las muestras en distintos estados internos.

Capítulo 4

Aplicaciones y Resultados

En este capítulo aplicaremos el RC clásico y cuántico a las distintas señales, explicadas en la sección 3.1, y mostraremos los resultados de ambas versiones para intentar mostrar que llegan a un resultado parecido y coherente. Haremos la demostración de esto con las señales sintéticas unidimensionales, que se pueden observar en las figuras 3.2 y 3.3, y posteriormente, analizaremos las señales EEG simuladas y reales para crear un clasificador binario que nos permita distinguir entre sujetos jóvenes y mayores.

4.1. RC en señal simple.

En esta sección, desarrollaremos nuestra primera prueba del RC clásico y cuántico con la señal sintética unidimensional más simple. La configuración del RC clásico va a constar de un número de neuronas en el reservorio de $N = 10$ y la longitud total de la señal de entrenamiento es de $T = 330$. Con respecto a la del RC cuántico, el sistema va a estar compuesto por $Q = 3$ qubits y los coeficientes α y β van a tomar los valores 0.05 y 0.95, respectivamente. Asimismo, al utilizarse la misma señal, la señal u_t tiene un tamaño de 330. Dicho esto, se pueden ver los tres primeros estados centrados respecto a la media del RC clásico y cuántico en la figura 4.1.

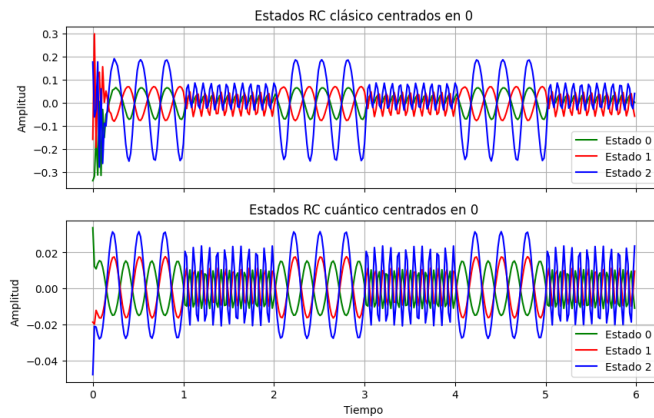


Figura 4.1: 3 primeros estados del RC clásico (arriba) y cuántico (abajo) en señal simple.

Se puede observar en la imagen 4.1 que los estados del RC cuántico captan el comportamiento de la señal desde el principio, mientras que en el clásico, al comienzo, cuesta

un poco. Para solucionarlo, previamente al cálculo de la envolvente de la señal, se van a eliminar los primeros puntos antes de que el reservorio se estabilice.

Por tanto, el siguiente paso es el cálculo de la envolvente usando la transformada de Hilbert, cuyo resultado se encuentra en la figura 4.2.

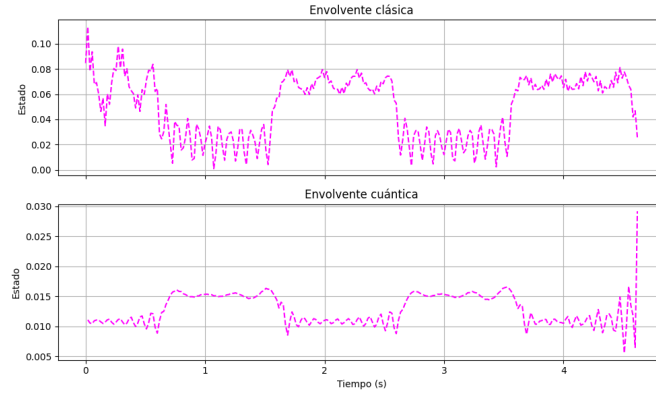


Figura 4.2: Envolvente cuántica y clásica en señal simple.

En la gráfica 4.2 se puede apreciar cómo ambas envolventes captan el cambio de frecuencia en la señal, que para este caso es uno de los principales objetivos. Por tanto, una vez tenemos la matriz de envolventes A , pasamos a la capa de salida de nuestro RC donde aplicaremos un *clustering* sobre los elementos de la matriz.

Acerca de la técnica de clustering a utilizar, para este ejemplo vamos a utilizar dos: *K-Means* y *Gaussian Mixture Model* (GMM). En primer lugar, vamos a definir brevemente estas dos técnicas:

- **K-Means:** algoritmo de clustering basado en la minimización de distancias euclidianas. Su objetivo es particionar n observaciones en K clústeres $C = \{C_1, C_2, \dots, C_K\}$ de forma que se minimice la suma de las distancias cuadradas entre los puntos y sus respectivos centroides μ_k :

$$\min_C \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \quad \text{donde } \mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

El algoritmo itera entre dos pasos: (1) asignar cada punto al centroide más cercano, y (2) recalcular cada centroide como el promedio de los puntos asignados a su clúster.

- **Gaussian Mixture Model (GMM):** modelo probabilístico que representa los datos como una *mezcla de distribuciones gaussianas*, permitiendo que cada punto pertenezca parcialmente a múltiples clústeres. Cada componente de la mezcla es una distribución normal multivariada:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k) \quad \text{donde } \sum_{k=1}^K \pi_k = 1$$

El algoritmo Expectation-Maximization (EM) estima los parámetros mediante iteraciones de dos pasos:

- **Expectation (E-step):** se calcula la probabilidad de cada componente k para cada punto x_i :

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)} \quad \text{donde } \gamma_{ik} \text{ es la probabilidad de que } x_i \text{ pertenezca al clúster } k$$

- **Maximization (M-step):** se actualizan los parámetros del modelo en base a estas probabilidades:

$$\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}} \quad \text{media ponderada,}$$

$$\Sigma_k = \frac{\sum_i \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top}{\sum_i \gamma_{ik}} \quad \text{matriz de covarianza ponderada,}$$

$$\pi_k = \frac{1}{n} \sum_i \gamma_{ik} \quad \text{peso de mezcla del componente } k$$

Este enfoque permite modelar clústeres no circulares a diferencia de K-Means, que suele suponer esa forma.

Para ambos algoritmos, se les va a indicar el número de clústeres requeridos y que en este caso va a ser 2, ya que tenemos 2 frecuencias distintas en la señal. En la figura 4.3 se puede ver el resultado del clustering del RC clásico y observamos que con ambos algoritmos conseguimos el mismo resultado y en la figura 4.4 se encuentra el clustering del RC cuántico en el que también se consigue el resultado para las dos técnicas.

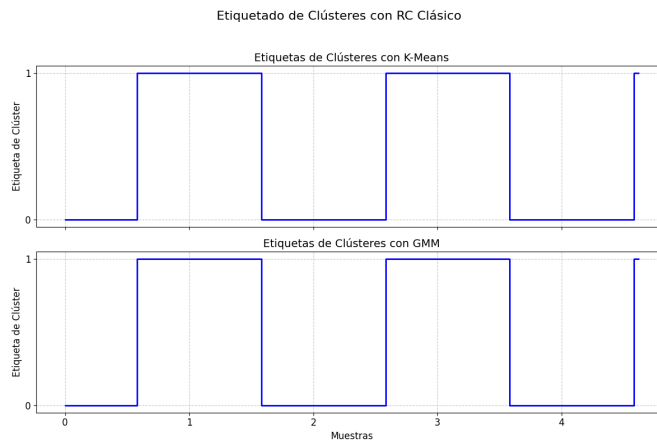


Figura 4.3: Clustering del RC clásico en señal simple con K-Means (arriba) y GMM (abajo).

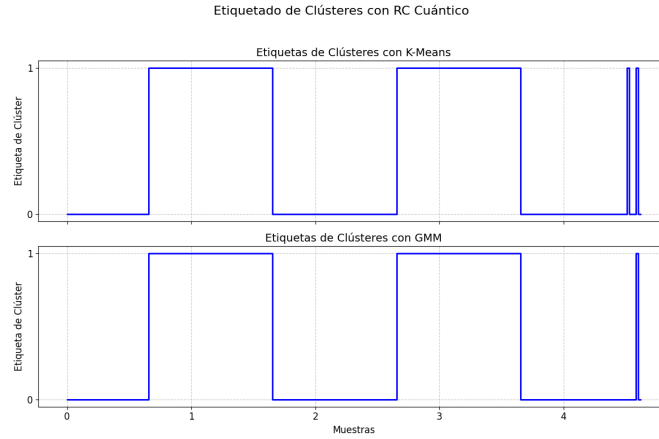


Figura 4.4: Clustering del RC cuántico en señal simple con K-Means (arriba) y GMM (abajo).

4.2. RC en señal compleja.

A continuación, vamos a desarrollar la segunda prueba del RC clásico y cuántico con la señal sintética unidimensional más compleja que se puede visualizar en la figura 3.3. El procedimiento va a ser muy parecido al visto en la sección 4.1 con algún añadido tanto para la versión clásica como cuántica.

Con respecto al RC clásico, el principal añadido es un filtro pasa-banda que permite el paso de señales dentro de un rango específico de frecuencias, atenuando aquellas que se encuentran fuera de dicho intervalo. En nuestro caso, puede ser útil para resaltar componentes frecuenciales relevantes. Esto puede mejorar la calidad de las representaciones temporales generadas por el reservoir y facilitar tareas posteriores como el clustering.

La principal novedad en el RC cuántico, aparte también de la adición del filtro pasa-banda, es la implementación de diferentes conectividades entre los qubits. En la versión comentada en la sección 3.3, se habla de entrelazamiento entre qubits consecutivos en forma de anillo, pues aparte de este tipo de conectividad se van a aplicar tres más. Estos tipos de conectividad son los siguientes:

- **Star (estrella):** donde un qubit central (el qubit 0) se conecta con todos los demás. Es útil para propagar información desde un nodo dominante al resto del sistema. Este esquema introduce un patrón jerárquico en el entrelazamiento.
- **Random (aleatoria):** se generan pares de qubits aleatorios y se aplica una puerta CNOT a una fracción de ellos ($\text{num_qubits} // 2$). Esto introduce diversidad estructural y una dinámica más caótica en el reservoir, lo cual puede enriquecer su capacidad de representación.
- **Full (total):** todos los pares posibles de qubits se entrelazan. Este tipo de conectividad maximiza la interacción entre los qubits, lo cual puede aumentar la capacidad expresiva del sistema cuántico.

Una vez explicados los cambios, cabe destacar las configuraciones del RC clásico y cuántico. El número de neuronas en el RC clásico es $N = 10$ y la longitud total de la señal

de entrenamiento es $T = 1760$. El RC cuántico consta de un sistema de $Q = 3$ qubits con los hiperparámetros α y β con valor de 0.05 y 0.95, respectivamente. Como tipo de conectividad entre los qubits, vamos a utilizar la configuración *full*. En la figura 4.5 se pueden ver los tres primeros estados de ambas versiones en la señal compleja. Se puede notar las cuatro frecuencias distintas de la señal.

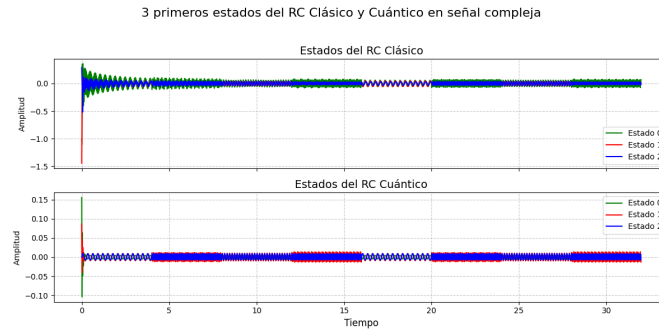


Figura 4.5: 3 primeros estados del RC clásico (arriba) y cuántico (abajo) en señal compleja.

El siguiente paso es el de calcular la matriz de envolventes, y cuyo resultado se puede observar en la figura 4.6. En esta figura, resalta con mayor claridad el cambio de frecuencia que ocurre en la señal y como ambas versiones son capaces de captarlo.

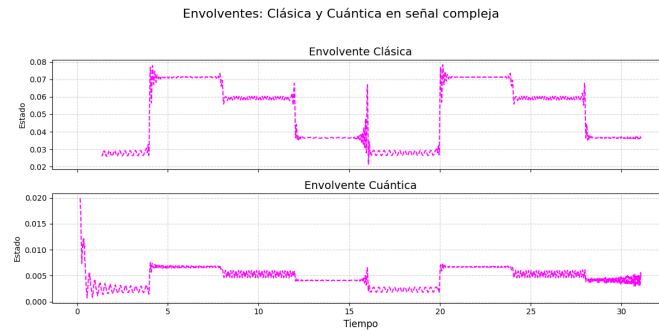


Figura 4.6: Envolvente clásica (arriba) y cuántica (abajo) en señal compleja.

Ahora, vamos a aplicar el clustering y en esta ocasión el único algoritmo que vamos a mostrar es el GMM. Esta decisión se basa en que esta técnica es generalmente más robusta que K-Means debido a su naturaleza probabilística. Mientras que K-Means asigna de forma estricta cada punto al centroide más cercano y es sensible a la inicialización de estos, GMM modela los datos como una combinación de distribuciones gaussianas, permitiendo que cada punto tenga una probabilidad de pertenencia a múltiples clústeres. Esta flexibilidad permite que GMM capture estructuras más complejas y clústeres con formas no necesariamente esféricas o de igual tamaño.

El resultado del clustering de la matriz de envolventes para ambas versiones se puede observar en la figura 4.7. Cabe recordar que el número de clústeres en esta ocasión es de cuatro, que se corresponde con las cuatro frecuencias distintas de la señal.

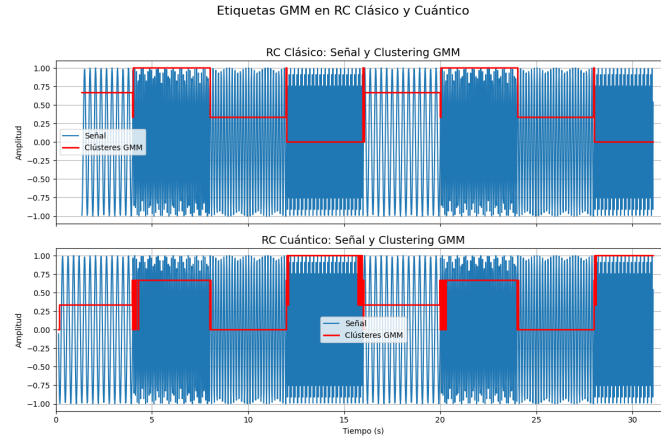


Figura 4.7: Clustering del RC clásico (arriba) y cuántico (abajo) en señal compleja con GMM.

Podemos identificar los distintos clústeres con los dos algoritmos de forma satisfactoria, hay que destacar que las etiquetas de los clústeres pueden ser distintas entre la versión clásica y la cuántica lo que no significa que el resultado esté mal, ya que lo importante es que las muestras de un clúster siempre se asignen al mismo.

Como última parte de esta sección, vamos a realizar una prueba con todos los tipos de conectividad entre qubits que explicamos anteriormente y así poder saber cuáles obtienen el resultado correcto. Para ello, se va a definir un sistema cuántico común con $Q = 3$ qubits y se va a aplicar cada una de las configuraciones (*full*, *ring*, *star* y *random*) sobre la señal compleja.

En la figura 4.8, podemos visualizar el clustering resultante.

Se observa que en las configuraciones *full*, *ring* y *star*, las transiciones entre estados son relativamente esporádicas y bien definidas, sobretodo en la configuración *full* y *star*. Estas transiciones están marcadas por saltos verticales en la gráfica, que representan los instantes en los que la señal cambia de un estado a otro.

Sin embargo, en el caso de la conectividad *random*, el comportamiento es diferente. La señal muestra una gran cantidad de transiciones rápidas y frecuentes entre estados, lo que genera una secuencia de etiquetas altamente variable. Esta variabilidad puede deberse a una mayor inestabilidad dinámica en el sistema generado con conectividad aleatoria.

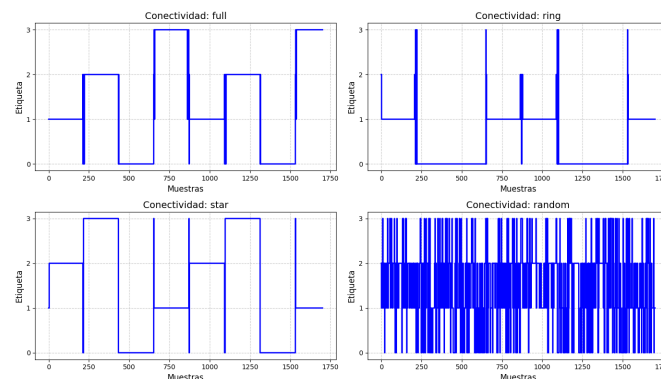


Figura 4.8: Clustering de cada una de las conectividades en la señal compleja con GMM.

4.3. Clasificador de señales EEG.

A continuación, vamos a describir en esta parte la aplicación del RC clásico y cuántico sobre señales EEG multidimensionales. En primer lugar, lo haremos con las señales EEG sintéticas y posteriormente, con las señales reales. Una vez tengamos los resultados del clustering para cada una de las señales, construiremos un clasificador que intentará distinguir entre personas jóvenes y mayores.

4.3.1. Señales EEG simuladas.

Las señales EEG simuladas, comentadas en la sección 3.1, nos van a servir como primera prueba de nuestro clasificador. Para la construcción de este, vamos a adaptar la estructura del RC y, una vez hecho esto, haremos una búsqueda de diferentes hiperparámetros para intentar encontrar la mejor configuración para cada una de las versiones. La evaluación de estas configuraciones se hará con la creación de un clasificador simple y se escogerán las que den mejor rendimiento. Por último, se harán más pruebas con diferentes modelos para intentar mejorar el resultado de la clasificación.

En primer lugar, hemos añadido a la estructura de nuestro RC una normalización espectral sobre la matriz de pesos recurrentes del reservorio W_{RC} . Esta normalización consiste en escalar dicha matriz para que su radio espectral, es decir, el valor absoluto del mayor autovalor de W_{RC} , sea igual a un valor deseado. En nuestro caso, hemos fijado este valor en 0,9. Esta elección no es arbitraria: si el radio espectral es demasiado alto, la dinámica interna del reservorio tiende a volverse inestable, con estados que crecen de forma descontrolada; mientras que si es demasiado bajo, el sistema pierde sensibilidad a las entradas y reduce significativamente su capacidad de memoria.

La normalización se lleva a cabo dividiendo la matriz W_{RC} por su radio espectral original y multiplicando por 0,9, es decir:

$$W_{RC} \leftarrow \frac{W_{RC}}{\rho(W_{RC})} \cdot \rho_{\text{deseado}}, \quad \text{con } \rho_{\text{deseado}} = 0,9$$

A continuación, realizaremos una búsqueda de la configuración del RC clásico y cuántico que dé mejor rendimiento en un clasificador simple común. El hiperparámetro a probar en el RC clásico es el número de neuronas en el reservorio, donde utilizaremos 10, 20, 50 y 100 neuronas, y en el RC cuántico, probaremos 2 y 5 qubits con conectividad full y star. El motivo de utilizar como máximo 5 qubits es debido a que los resultados más satisfactorios durante este proceso han sido con este número de qubits.

La técnica de clustering a utilizar es el GMM. En las señales anteriores, se aplicaba este algoritmo con los parámetros por defecto y únicamente se especificaba el número de clusters. Ahora, el proceso comienza con una etapa de preprocesamiento en la que los datos son normalizados utilizando `StandardScaler`. Posteriormente, se inicializa el modelo GMM configurando los siguientes parámetros: el número de clústeres, el número de inicializaciones fijado en 10 para mejorar la robustez frente a mínimos locales, el número máximo de iteraciones del algoritmo EM establecido en 500, y una tolerancia de convergencia de 10^{-3} .

Cabe destacar que, en el análisis de señales EEG [26], es común considerar la existencia de cuatro estados cerebrales diferenciados (sueño profundo, somnolencia, vigilia relajada

y alerta). Por ello, con estas señales se ha fijado el número de clústeres del modelo GMM en $n = 4$.

Con el fin de mejorar la capacidad del clasificador, haremos una extracción de características a partir del *clustering* realizado. Para cada una de las secuencias de etiquetas de *clustering* obtenidas (un total de 40 señales), se extraen tres grupos principales de características. Estas se dividen en dos tipos: **características específicas por estado** y **características globales de la dinámica de transición**.

- **Distribución de frecuencias por estado:** se calcula un histograma normalizado que representa la proporción del tiempo que la señal permanece en cada uno de los $n = 4$ estados. Esta representación permite caracterizar la presencia relativa de cada estado cerebral a lo largo del tiempo. Se trata de una característica por estado, ya que proporciona una proporción específica para cada uno de los clústeres.
- **Duración media por estado:** se calcula el número promedio de pasos consecutivos que la señal permanece en un mismo estado antes de cambiar a otro. Para ello, se agrupan las etiquetas secuenciales iguales y se calcula la longitud media de estas agrupaciones por cada estado. Esta también es una característica por estado, debido a que refleja la estabilidad temporal individual de cada uno de los clústeres.
- **Matriz de transición entre estados:** se construye una matriz cuadrada de tamaño $n \times n$ que contabiliza la frecuencia con la que se producen transiciones entre pares de estados consecutivos. Esta matriz se normaliza dividiendo por el total de transiciones, y finalmente se aplanan sus valores para obtener un vector de características. Dado que representa el patrón completo de transiciones entre todos los estados, se considera una característica global de la secuencia.

Para llevar a cabo esta extracción de características en Python, se han utilizado las siguientes bibliotecas estándar:

- NumPy, para el manejo de vectores, histogramas, medias y operaciones matriciales.
- `itertools`, específicamente la función `groupby`, para agrupar etiquetas consecutivas iguales en la secuencia y calcular la duración media por estado.

Una vez hecho todo esto, podemos pasar a la construcción del clasificador simple que va a ser un modelo **Random Forest**. Este es un modelo basado en el método de *bagging*, que entrena múltiples árboles de decisión sobre subconjuntos aleatorios de los datos para mejorar la generalización y reducir el sobreajuste.

Dentro de sus hiperparámetros, cabe destacar:

- **n_estimators:** Especifica el número de árboles que componen el bosque aleatorio. Un mayor número de árboles puede mejorar la precisión del modelo, aunque también aumenta el coste computacional. Su valor por defecto es 100.
- **max_depth:** Controla la profundidad máxima permitida para cada árbol. Si no se especifica, los árboles crecerán hasta que todas las hojas sean puras o contengan menos de `min_samples_split` instancias.

Para este caso, utilizaremos un Random Forest con todos sus valores por defecto que se va a entrenar con el 70 % del conjunto de entrada y la predicción va a realizarse sobre el 30 % restante.

Los resultados de las configuraciones del RC clásico se pueden observar en la tabla 4.1 y del RC cuántico en la tabla 4.2.

Tabla 4.1: Resultados del clasificador Random Forest básico sobre el RC clásico.

Neuronas	Accuracy	F1 (Joven)	F1 (Mayor)	F1-macro	F1-weighted
10	0.67	0.60	0.71	0.66	0.65
20	0.42	0.22	0.53	0.38	0.35
50	0.83	0.86	0.80	0.83	0.83
100	0.58	0.55	0.62	0.58	0.57

Tabla 4.2: Resultados del clasificador Random Forest básico sobre el RC cuántico.

Configuración	Accuracy	F1 (Joven)	F1 (Mayor)	F1-macro	F1-weighted
5 qubits, star	0.92	0.93	0.89	0.91	0.91
5 qubits, full	0.75	0.73	0.77	0.75	0.74
2 qubits, star	0.33	0.33	0.33	0.33	0.33
2 qubits, full	0.50	0.40	0.57	0.49	0.47

Las métricas de clasificación que hemos utilizado son: *accuracy*, *f1*, *f1-macro* y *f1-weighted*. Sus características principales son las siguientes:

- **Accuracy:** proporción de predicciones correctas sobre el total.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

TP: verdaderos positivos, TN: verdaderos negativos, FP: falsos positivos, FN: falsos negativos.

- **F1 (Joven):** evalúa qué tan bien el modelo identifica correctamente las señales EEG de jóvenes, considerando tanto los verdaderos positivos como los falsos positivos y negativos.

$$F1_{\text{Joven}} = 2 \cdot \frac{P_{\text{Joven}} \cdot R_{\text{Joven}}}{P_{\text{Joven}} + R_{\text{Joven}}}$$

P: precisión(relación entre los positivos identificados correctamente y todos los positivos identificados).
R: recall(relación entre los verdaderos positivos previstos y lo que realmente se ha etiquetado.)

- **F1 (Mayor):** evalúa el desempeño del modelo al clasificar correctamente señales EEG de mayores.

$$F1_{\text{Mayor}} = 2 \cdot \frac{P_{\text{Mayor}} \cdot R_{\text{Mayor}}}{P_{\text{Mayor}} + R_{\text{Mayor}}}$$

- **F1-macro:** promedio no ponderado de los F1-scores por clase.

$$F1_{\text{macro}} = \frac{F1_{\text{Joven}} + F1_{\text{Mayor}}}{2}$$

- **F1-weighted:** promedio ponderado de los F1-scores según el número de muestras por clase.

$$F1_{\text{weighted}} = \frac{n_{\text{Joven}} \cdot F1_{\text{Joven}} + n_{\text{Mayor}} \cdot F1_{\text{Mayor}}}{n_{\text{total}}}$$

De los resultados obtenidos en las tablas 4.1 y 4.2, consideramos que la mejor configuración del RC clásico es con 50 neuronas en el reservorio y la del RC cuántico es con conectividad star y 5 qubits.

Como siguiente paso, vamos a intentar mejorar los resultados de clasificación haciendo uso de distintos modelos. Para ello, vamos a probar distintas configuraciones de **Random Forest**, de **Regresión Logística** y de **XGBoost**.

La regresión logística modela la probabilidad de una clase mediante la función sigmoide aplicada a una combinación lineal de las variables de entrada. En **scikit-learn**, el parámetro **C** controla la fuerza de la regularización inversamente: valores pequeños de **C** implican una regularización más fuerte. El parámetro **penalty** especifica el tipo de regularización aplicada, como **l1** (Lasso) o **l2** (Ridge), que ayudan a prevenir el sobreajuste.

Por otro lado, XGBoost (Extreme Gradient Boosting) es un algoritmo basado en árboles de decisión optimizados mediante técnicas de *boosting* por gradiente. Utiliza una estrategia aditiva para construir modelos fuertes combinando muchos modelos débiles, mejorando progresivamente los errores de las predicciones anteriores. Destaca por su eficiencia, regularización incorporada y buen rendimiento.

Además, aparte de GMM, vamos a probar otra técnica de clustering llamada Spectral Clustering. Este algoritmo de agrupamiento utiliza los autovectores del grafo de similitud entre los datos para realizar la partición. Asimismo, permite identificar clústeres no lineales mediante una representación espectral del espacio de datos. Este enfoque es especialmente útil cuando las clases tienen formas complejas o no convexas.

En la tabla 4.3, se pueden visualizar los resultados del RC clásico para cada algoritmo de clustering y para cada modelo, indicando además los mejores parámetros de cada uno de estos.

Tabla 4.3: Resultados de clasificación con diferentes modelos y técnicas de clustering con RC clásico.

Clustering	Modelo	Accuracy	F1	F1-macro	F1-weighted	Mejores parámetros
GMM	Random Forest	0.67	0.71	0.66	0.65	max_depth=2, min_samples_split=2, n_estimators=50
	Logistic Regression	0.67	0.67	0.67	0.67	C=0.01, penalty='l2'
	XGBoost	0.50	0.57	0.49	0.47	max_depth=2, n_estimators=10, reg_alpha=1, reg_lambda=10
Spectral	Random Forest	0.42	0.53	0.38	0.40	max_depth=None, min_samples_split=2, n_estimators=50
	Logistic Regression	0.50	0.57	0.49	0.50	C=1, penalty='l2'
	XGBoost	0.42	0.53	0.38	0.40	max_depth=2, n_estimators=10, reg_alpha=1, reg_lambda=10

Dado que nuestro conjunto de entrada no es muy grande, con 40 señales y 24 caracte-

rísticas, es recomendable usar modelos con baja complejidad y regularización adecuada. En el caso del clasificador Random Forest, se han considerado valores bajos para el número de árboles (`n_estimators`) como 10, 20 y 50, junto con una profundidad máxima limitada (`max_depth`). Además, el parámetro `min_samples_split` controla la cantidad mínima de muestras necesarias para dividir un nodo.

Para regresión logística, se han incluido valores pequeños del hiperparámetro `C`, como 0.01, 0.1 y 1. Se ha utilizado exclusivamente la penalización de tipo 12.

En cuanto al modelo **XGBoost**, también se han seleccionado configuraciones poco complejas: `n_estimators` pequeños (10 y 20), profundidades de árbol bajas (`max_depth` de 2 o 3), y se ha explorado la regularización mediante `reg_lambda` (L2) y `reg_alpha` (L1).

Sin embargo, los resultados obtenidos en la tabla 4.3 no mejoran al clasificador Random Forest básico definido en el cuadro 4.1 con 50 neuronas.

Con respecto al RC cuántico, los resultados se pueden consultar en la tabla 4.4. Es cierto que, en este caso, tampoco mejora los resultados del Random Forest básico, pero sí que se puede observar que los resultados son relativamente superiores a los del RC clásico y se puede considerar que, en general, funciona mejor. Asimismo, el algoritmo GMM funciona considerablemente mejor que el Spectral Clustering.

Tabla 4.4: Resultados de clasificación con diferentes modelos y técnicas de clustering con RC cuántico.

Clustering	Modelo	Accuracy	F1	F1-macro	F1-weighted	Mejores parámetros
GMM	Random Forest	0.75	0.77	0.75	0.75	<code>max_depth=3,</code> <code>min_samples_split=5,</code> <code>n_estimators=50</code>
	Logistic Regression	0.83	0.86	0.83	0.83	<code>C=0.01, penalty='l2'</code>
	XGBoost	0.75	0.77	0.75	0.75	<code>max_depth=2,</code> <code>n_estimators=10,</code> <code>reg_alpha=1,</code> <code>reg_lambda=10</code>
Spectral	Random Forest	0.25	0.20	0.20	0.23	<code>max_depth=2,</code> <code>min_samples_split=2,</code> <code>n_estimators=50</code>
	Logistic Regression	0.25	0.24	0.24	0.26	<code>C=0.01, penalty='l2'</code>
	XGBoost	0.25	0.20	0.20	0.23	<code>max_depth=3,</code> <code>n_estimators=10,</code> <code>reg_alpha=1,</code> <code>reg_lambda=10</code>

Por último, en las figuras 4.9 y 4.10 se pueden visualizar las curvas de aprendizaje del mejor modelo obtenido del RC clásico y del RC cuántico, respectivamente. Estas gráficas nos permiten ver la pérdida del conjunto de entrenamiento y de validación para distintos tamaños del conjunto de entrenamiento. La particularidad de la forma de las gráficas se corresponde con el reducido tamaño de nuestro conjunto de entrada.

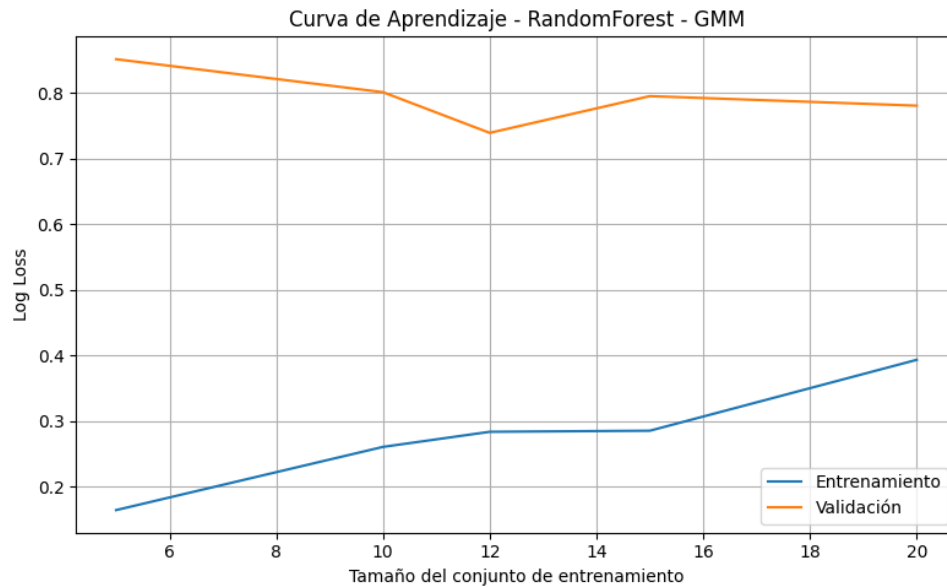


Figura 4.9: Curva de aprendizaje del mejor modelo obtenido del RC clásico en señales EEG simuladas.

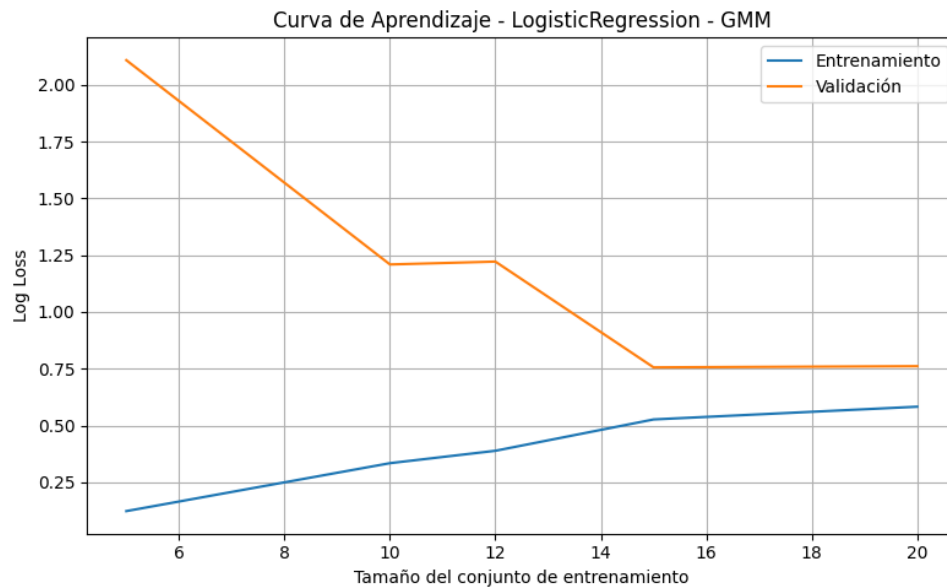


Figura 4.10: Curva de aprendizaje del mejor modelo obtenido del RC cuántico en señales EEG simuladas.

La función de coste que hemos utilizado en estas curvas de aprendizaje es la `log_loss` que se encarga de evaluar la calidad de las predicciones basándose en las probabilidades estimadas para cada clase. A diferencia de la simple exactitud, esta función penaliza con mayor severidad las predicciones seguras pero incorrectas. Durante el proceso de aprendizaje, se utiliza la versión negativa de esta métrica, `neg_log_loss`, porque las funciones están diseñadas para maximizar (y no minimizar) los valores durante el proceso de validación cruzada.

4.3.2. Señales EEG reales.

Una vez han sido hechas todas las pruebas explicadas durante el capítulo 4, podemos pasar a la aplicación de las señales EEG reales a nuestras versiones de RC. Estas han sido explicadas en el apartado 3.1, y cabe destacar que, disponemos de 47 señales en total, 23 de jóvenes y 24 de mayores, y cada una contiene 64 canales.

La estructura del RC clásico y cuántico es la misma que la vista en 4.3.1, por lo que tenemos 50 neuronas en el reservorio en la parte clásica y en la cuántica utilizamos la conectividad star y 5 qubits. El algoritmo de clustering a utilizar va a ser el GMM debido a que ha sido el que mejor ha funcionado durante este trabajo.

En la tabla 4.5, se pueden observar los resultados de la clasificación haciendo uso de un Random Forest básico similar al explicado en la sección anterior. El rendimiento en este caso del RC clásico es superior a la versión cuántica.

Tabla 4.5: Comparación del rendimiento del modelo Random Forest entre el RC clásico y el RC cuántico.

Tipo de RC	Accuracy	F1 (Joven)	F1 (Mayor)	F1-macro	F1-weighted
Clásico	0.67	0.62	0.71	0.66	0.67
Cuántico	0.53	0.63	0.36	0.50	0.47

En las tablas 4.6 y 4.7, se pueden consultar los resultados de la versión clásica y cuántica probando diferentes configuraciones de los modelos Random Forest, Regresión Logística y XGBoost.

Tabla 4.6: Resultados de los clasificadores de señales EEG con RC clásico.

Modelo	Mejores parámetros	Accuracy	F1-macro	F1-weighted	F1
Random Forest	{max_depth:2, min_samples:5, n_estimators:50}	0.53	0.50	0.52	0.63
Logistic Regression	{C:1, penalty:'l2'}	0.53	0.50	0.52	0.63
XGBoost	{max_depth=2, n_estimators=10, reg_alpha=1, reg_lambda=1}	0.53	0.53	0.54	0.59

Tabla 4.7: Resultados de los clasificadores de señales EEG con RC cuántico.

Modelo	Mejores parámetros	Accuracy	F1-macro	F1-weighted	F1
Random Forest	{max_depth: 2, min_samples: 5, n_estimators: 20}	0.47	0.46	0.46	0.46
Logistic Regression	{C: 0.01, penalty: 'l2'}	0.67	0.66	0.65	0.71
XGBoost	{max_depth: 2, n_estimators: 10, reg_alpha: 1, reg_lambda: 10}	0.27	0.25	0.23	0.27

Los resultados del RC clásico y cuántico en las señales EEG reales son bastante similares debido en parte a la complejidad del problema. Cabe destacar la mejora del RC cuántico probando más modelos lo que sigue observándose ventaja cuántica. Asimismo, el modelo que mejor funciona es la Regresión Logística, lo que significa que en entornos ruidosos como los de las señales reales un modelo sencillo puede ser más adecuado.

En las figuras 4.11 y 4.12, podemos observar las curvas de aprendizaje del mejor modelo para cada versión de una manera similar a las gráficas 4.9 y 4.10 usando la función de coste `neg_log_loss`. El proceso de aprendizaje sigue siendo no muy limpio ya que el tamaño del conjunto de entrenamiento no es demasiado grande.

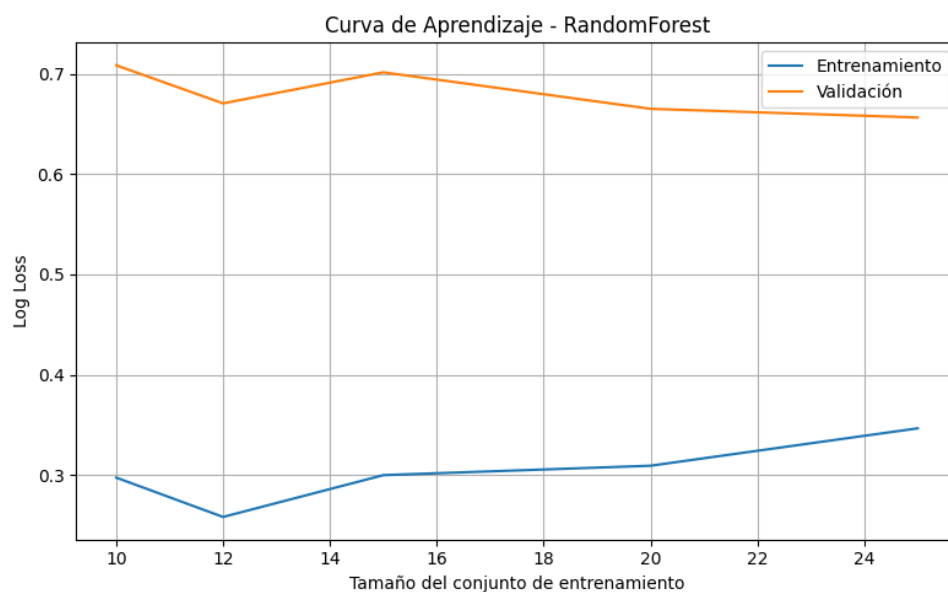


Figura 4.11: Curva de aprendizaje del mejor modelo obtenido del RC clásico en señales EEG reales.

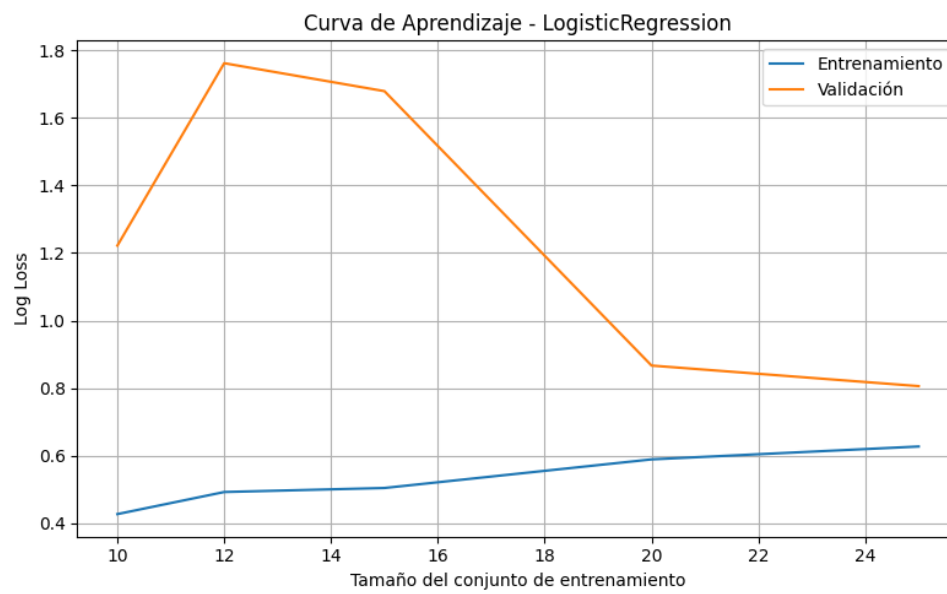


Figura 4.12: Curva de aprendizaje del mejor modelo obtenido del RC cuántico en señales EEG reales.

Capítulo 5

Conclusiones y proyección futura.

5.1. Conclusiones

Después de los resultados vistos durante el Capítulo 4, podemos concluir con que las versiones de RC clásico y cuántico funcionan satisfactoriamente con la señal simple y compleja, el clustering es el correcto y hemos sido capaces de conseguirlo con dos algoritmos diferentes como K-Means y GMM. En cuanto al RC cuántico, hemos demostrado que las conectividades full y star consiguen también resolver el problema.

Por otro lado, con las señales EEG simuladas hicimos diversas pruebas para encontrar una configuración adecuada para cada una de las versiones y pudimos llegar a unos resultados bastante correctos. La extracción de características del clustering ha sido un aspecto clave para que el clasificador tuviera más información y pudiera clasificar correctamente. Es cierto que, el modelo que mejor ha funcionado ha sido el Random Forest básico con ambas versiones, lo cual se puede pensar que los modelos más refinados no han conseguido funcionar como se esperaba pero se puede notar que la diferencia tampoco es muy amplia. Hay que tener en cuenta que los resultados de predicción son de apenas 12 personas. Con estas señales, también se ha confirmado que el mejor algoritmo de clustering para este problema es el GMM a pesar de que teóricamente, el Spectral Clustering podría ser una buena opción para este tipo de problemas.

En cuanto a las señales EEG reales, se puede observar que los resultados son relativamente inferiores a las señales sintéticas y seguramente esto sea debido a la naturalidad de los datos. Las señales sintéticas tienen menos ruido y su complejidad es inferior a datos reales donde además, no se le han aplicado un preprocesamiento exhaustivo. La idea de este proyecto era también que el RC fuera capaz de detectar el comportamiento de las señales sin un trabajo previo muy grande.

Por último, el objetivo principal de este trabajo era la demostración de que la versión cuántica de RC obtiene unos resultados similares e incluso superiores a la versión clásica y parece que se ha cumplido. En este trabajo se ha demostrado que en las señales sintéticas el RC cuántico funciona considerablemente mejor que el clásico y en las señales reales, los resultados son algo más parecidos pero se puede notar la mejoría cuántica. Asimismo, los resultados de este TFG abren la posibilidad de implementar los algoritmos de RC cuántico en hardware cuántico real, lo que además permitiría escalar a un mayor número de qubits, que presumiblemente conduciría a unos mejores resultados.

5.2. Trabajo futuro

Este trabajo puede ser extendido en varias direcciones que permitan una mejora en la capacidad de generalización y precisión del sistema. En primer lugar, se propone aumentar el número de qubits en el sistema de QRC, con el objetivo de explorar si una mayor dimensionalidad del espacio de estados cuánticos puede mejorar el rendimiento del modelo.

En este sentido, se empleó la transformada de Hilbert para obtener la envolvente de las señales, por lo que sería interesante investigar otras técnicas de extracción de envolvente, como el método de detección de picos, el análisis por energía instantánea, o el uso de filtros pasa banda adaptativos. El objetivo sería determinar si alguna de estas alternativas permite mejorar la calidad de las características extraídas.

Por último, en la capa de salida del RC, se plantea explorar el uso de algoritmos de clustering alternativos, siendo el *Quantum Clustering* [27] una opción prometedora. Este enfoque podría ofrecer una mayor robustez frente al ruido y a las complejidades de los datos EEG, al aprovechar estructuras no lineales de alta dimensión presentes en el espacio de características generado por el reservoir.

Adicionalmente, se propone implementar el sistema de QRC sobre hardware cuántico real, con el fin de evaluar su viabilidad práctica y su comportamiento frente a las limitaciones físicas de los dispositivos clásicos. También sería relevante aplicar el RC cuántico a otros conjuntos de datos EEG, orientados a diferentes tareas cognitivas o clínicas, para estudiar la capacidad de generalización del modelo en diferentes contextos.

Bibliografía

- [1] S. Egozcue R.M. Pabon M.T. Alonso F. Ramos-Arguelles, G. Morales. Técnicas básicas de electroencefalografía: principios y aplicaciones clínicas. *Anales del Sistema Sanitario de Navarra*, 32:69 – 82, 00 2009.
- [2] Gouhei Tanaka, Toshiyuki Yamane, Jean Benoit Héroux, Ryosho Nakane, Naoki Kanazawa, Seiji Takeda, Hidetoshi Numata, Daiju Nakano, and Akira Hirose. Recent advances in physical reservoir computing: A review. *Neural Networks*, 115:100–123, 2019.
- [3] Lyudmila Grigoryeva and Juan-Pablo Ortega. Echo state networks are universal, 2018.
- [4] Pavithra Koralalage, Ireoluwa Fakeye, Pedro Machado, Jason Smith, Isibor Kennedy Ihianle, Salisu Wada Yahaya, Andreas Oikonomou, and Ahmad Lotfi. Dynamic training of liquid state machines, 2023.
- [5] Benjamin Schrauwen, David Verstraeten, and Jan Campenhout. An overview of reservoir computing: Theory, applications and implementations. *Proceedings of the 15th European Symposium on Artificial Neural Networks*, pages 471–482, 01 2007.
- [6] Peter Dominey, Michael Arbib, and Jean-Paul Joseph. A model of corticostriatal plasticity for learning oculomotor associations and sequences. *Journal of Cognitive Neuroscience*, 7(3):311–336, 07 1995.
- [7] Dean V. Buonomano and Michael M. Merzenich. Temporal information transformed into a spatial code by a neural network with realistic properties. *Science*, 267(5200):1028–1030, 1995.
- [8] Wolfgang Maass, Thomas Natschläger, and Henry Markram. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11):2531–2560, 11 2002.
- [9] Herbert Jaeger and Harald Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667):78–80, 2004.
- [10] Sanjib Ghosh, Andrzej Opala, Michał Matuszewski, Tomasz Paterek, and Timothy C. H. Liew. Quantum reservoir processing. *npj Quantum Information*, 5(1), April 2019.
- [11] Makoto Negoro, Kosuke Mitarai, Keisuke Fujii, Kohei Nakajima, and Masahiro Kitagawa. Machine learning with controllable quantum dynamics of a nuclear spin ensemble in a solid, 2018.

- [12] Jiayin Chen, Hendra I. Nurdin, and Naoki Yamamoto. Temporal information processing on noisy quantum computers. *Physical Review Applied*, 14(2), August 2020.
- [13] Michael te Vrugt. An introduction to reservoir computing. In Michael te Vrugt, editor, *Artificial Intelligence and Intelligent Matter*. Springer Nature, 2025.
- [14] Robin M. Schmidt. Recurrent neural networks (rnns): A gentle introduction and overview, 2019.
- [15] Matteo Cucchi, Steven Abreu, Giuseppe Ciccone, Daniel Brunner, and Hans Klee-
mann. Hands-on reservoir computing: a tutorial for practical implementation. *Neu-
romorphic Computing and Engineering*, 2(3):032002, aug 2022.
- [16] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum
Information: 10th Anniversary Edition*. Cambridge University Press, 2010.
- [17] G.P. Berman, G.D. Doolen, R. Mainieri, and V.I. Tsifrinovich. *Introduction To Quan-
tum Computers*. World Scientific Publishing Company, 1998.
- [18] Fabrizio Cleri. Quantum computers, quantum computing and quantum thermody-
namics, 2024.
- [19] Pradeep Lamichhane and Danda B. Rawat. Quantum machine learning: Recent
advances, challenges and perspectives. *IEEE Access*, pages 1–1, 2025.
- [20] IBM Quantum. Qiskit api reference, 2024.
- [21] Google Quantum AI. Cirq: A python framework for creating, editing, and invoking
noisy intermediate scale quantum (nisq) circuits, 2024.
- [22] TensorFlow Quantum. Tensorflow quantum, 2024.
- [23] Xanadu. PennyLane: A quantum machine learning library, 2024.
- [24] SquLearn Developers. Sqlearn: Scikit-learn-inspired quantum machine learning fra-
mework, 2024.
- [25] Youssef Kora, Hadi Zadeh-Haghighi, Terrence C Stewart, Khabat Heshami, and
Christoph Simon. Frequency- and dissipation-dependent entanglement advantage
in spin-network quantum reservoir computing, 2024.
- [26] Yolanda Vives-Gilabert, Felipe Torres, Andre Gómez-Lombardi, and Wael El-Deredy.
Dynamics brain-state allocation using echo state network. In *Abstract Book 1: OHBM
2024 Annual Meeting*, volume 4. Organization for Human Brain Mapping, June 2024.
Poster presented at the OHBM 2024 Annual Meeting.
- [27] Raúl V. Casaña-Eslava, Paulo J. G. Lisboa, Sandra Ortega-Martorell, Ian H. Jarman,
and José D. Martín-Guerrero. A probabilistic framework for quantum clustering,
2019.